

VizTest: Optimal Confidence Intervals for Visual Testing

by William Poirier and David A. Armstrong

Abstract When presented with several confidence intervals plotted together, readers have a natural inclination to try to compare the bounds visually to test for the statistical significance of their differences. Comparing confidence bounds this way to perform a pairwise significance test yields erroneous results. The VizTest R package provides researchers with optimal confidence intervals for visual representation of pairwise tests. It is natively compatible with basic regression model outputs and includes a function that creates usable objects from other outputs, such as: 1) multilevel regression; 2) predicted probabilities; and 3) descriptive quantities. Pairwise tests are performed natively via normal theory or simulations, but users can choose to provide the results of alternative procedures directly. Multiplicity adjustments are natively implemented. We provide thorough examples of these different applications to facilitate the integration of the package into the users' workflows.

1 Introduction

Routinely, researchers convey the findings of their research through the visual display of estimates and confidence intervals (Few, 2012; Kastelec and Leoni, 2007). When readers engage these displays, they may naturally wonder whether estimates depicted in the visualization are statistically different from each other. Unfortunately, comparing the (non-)overlap of 95% confidence intervals does not equate to a test of difference at the 5% level. This is a problem that has been well understood for nearly a century. Both Dice and Laraas (1936) and Simpson and Roe (1939) recognized that $(1 - \alpha) \times 100\%$ confidence intervals did not naturally equate to tests at the α level. Their scholarship tried to identify rules of thumb for identifying differences that were either significant or not depending on the relative size of the standard errors of the estimates. Browne (1979) developed the same type of rules for other displays (e.g., $\pm 1SE$ bars) and provided more formalization around the problem of differing sampling variances. Tukey (1991) recognized that under the relatively restrictive assumption of equal standard errors and uncorrelated estimates 84% confidence intervals will overlap roughly 95% of the time under the null hypothesis. Goldstein and Healey (1995) proposed a more general approach that would identify the appropriate confidence level for tests at any level under the same assumptions adopted by Tukey. Payton et al. (2003) extends this idea to the non-normal context. Papers describing methods for solving this problem have been cited over 7500 times across the medical, natural and social sciences.

The scholarship until 2025 had focused on identifying the appropriate inferential confidence level for a pair of intervals. The idea was to identify the confidence level (γ) such that confidence intervals for estimates with the same standard errors as the ones under investigation would overlap under the null hypothesis with probability α . Tukey's 84% confidence interval is such an example for estimates with standard errors of roughly equal size. Displays of estimates often have several intervals to be evaluated. It is unlikely that any single level γ will be optimal for all pairs of intervals. If it is not, then different tests will be performed with different type I error rates. If the only way to produce valid visual tests is that the confidence intervals overlap with the desired type I error rate under the null hypothesis, there is no general solution.

In Armstrong II and Poirier (2025), we propose a novel solution to this problem that identifies the confidence level(s) that best represent the results of *all* pairwise tests between estimates. This paper describes the **VizTest** package that implements this procedure in R. The package is natively compatible with any object that has `coef` and `vcov` methods and whose pairwise tests are approximately accurate under the assumption of normality. This includes most regression model outputs. In other cases, there is a function that creates an object that is compatible with the package's main function from predicted probabilities, descriptive statistics and simulation output, to name a few.

As motivation, consider the following simple regression model using the built-in `chickwts` dataset. We regress weight of the chicks on the type of feed they consumed. We plot the predicted weights for each feed type with 95% confidence intervals in Figure 1. The confidence intervals do not convey information that is useful for pairwise testing just based on the (non-)overlaps in the confidence intervals. Of course, intervals that do not overlap indicate estimates that are statistically different from each other. However, the converse is not true.

Our solution involves searching for confidence levels such that overlapping confidence intervals indicate estimates that are not statistically different from each other and non-overlapping intervals indicate estimates that are statistically different from each other. We describe the algorithm below and then describe the **VizTest** package that implements it. We then provide several use cases that

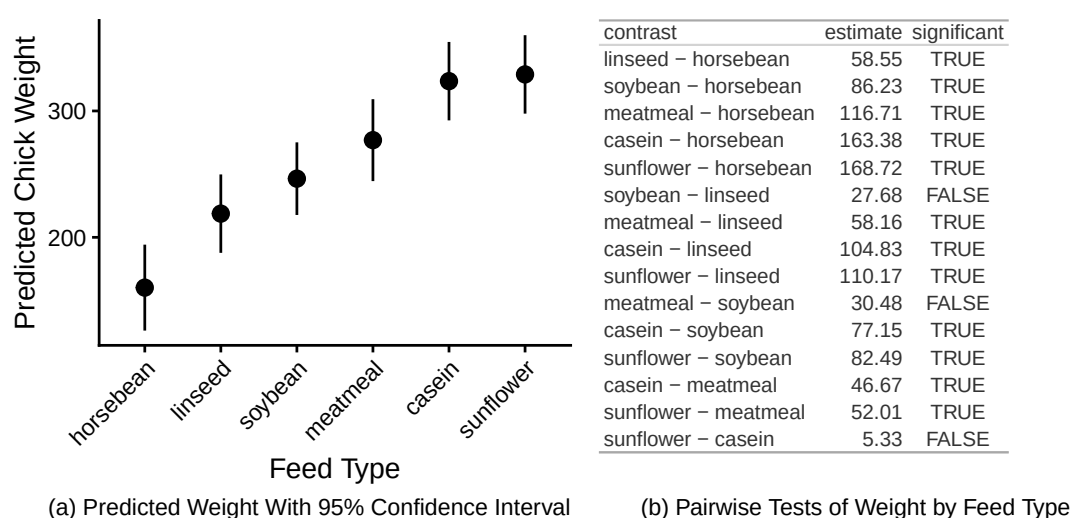


Figure 1: Predicted Chick Weights and Pairwise Tests. Panel (a) shows points that represent predicted chick weight with 95 percent confidence intervals represented with lines. Panel (b) shows pairwise tests (without multiplicity adjustment) for the pairs of feed types. The 95 percent intervals cannot be used to do hypothesis tests of difference at the .05 level here. While the sunflower and meatmeal intervals overlap, the pairwise differences suggest that they are nonetheless significantly different from each other.

demonstrate how to use the package in a variety of analytical situations. We compare our solution with alternative visualizations (compact letter displays, significance brackets, heatmaps, and comparison arrows) in the supplementary material, highlighting advantages and drawbacks for each.

2 Searching for inferential confidence intervals

All previous solutions, up to and including the most recent intervention in this literature (Radean, 2023) use the probability of overlap in the intervals to do the test. That is, the visualization and the test are inextricably linked. We propose a solution that estimates pairwise tests of all estimates and then performs a grid search over a set of predefined, reasonable values to find a confidence level γ , such that (non-)overlaps in the $\gamma \times 100\%$ confidence intervals correspond as closely as possible with the (in)significant differences. That is, we decouple the visualization from the testing. The algorithm we propose and describe in Armstrong II and Poirier (2025) is as follows:

- 1. Conduct all pairwise tests between estimates.** In the cases of estimates with a variance-covariance matrix, use normal theory to produce pairwise tests of the differences among the estimates. These tests can be adjusted for multiplicity or the user can supply a robust variance-covariance matrix if desired. In the case of simulation output, where the estimates are a number of draws $\times$ number of parameters matrix, the tests are done by calculating a matrix with differences between the vector of draws for each pair of estimates. The probability that the larger estimate is larger than the smaller estimate is then calculated and deemed “significant” if that probability is greater than $1 - \alpha$. This procedure produces a vector, s_{ij} that indicates whether estimate b_i is significantly different from estimate b_j (in which case $s_{ij} = 1$, or zero otherwise).

- 2. Find the inferential confidence level(s).** With the results from all the pairwise tests in s , we search over a range of values of γ to find the $(1 - \gamma)$ confidence interval(s)¹ such that the intervals for pairs of estimates that are significantly different from each other do not overlap and pairs of estimates that are not significantly different from each other do overlap.

- 3. Pick the inferential confidence level that is most useful.** If multiple γ are found to be equivalent in step 2, we should try to identify which is “best.” While we discuss some alternatives below, the idea is that it should be as easy as possible to do two things – identify where intervals do not overlap for those estimates that are statistically different from each other and identify where intervals do overlap for those estimates that are not statistically different from each other.

If a γ level is identified where **all** the pairwise tests match the visual tests, then using that level in the corresponding visualization would solve the aforementioned issue – readers would be able to

¹We call $(1 - \gamma)$ the inferential confidence level, the confidence level that best visually represents the underlying pairwise tests.

easily identify which pairs of estimates are statistically distinguishable from each other and which are not. This is implemented in the **VizTest** package described below along with several use cases that will allow researchers to deploy the algorithm described above in a wide array of real-world analytical situations.

3 The **VizTest** package

The **VizTest** package allows the user to find the optimal confidence level for visual testing in many different scenarios. The `viztest()` function is a generic that currently dispatches one of two methods – the default method which performs a normal theory test for pairwise difference and the `vtsim` method which operates on realized samples from a distribution (e.g., a matrix of samples from a Bayesian posterior distribution). It currently accepts the following arguments:

- `obj`: the object that contains the estimates.
- `test_level`: the type 1 error rate (α) for the pairwise tests (default at 0.05).
- `range_levels`: the range of γ to try (default from 0.25 to 0.99).
- `level_increment`: step size between values of `range_levels` (default at 0.01).
- `adjust`: multiplicity adjustment (`holm`, `hochberg`, `hommel`, `bonferroni`, `BH`, `BY`, `fdr` or `none`) to use when computing the p -values for the pairwise tests. This is not implemented for the `sim` method. The default is to do no adjustment.²
- `cifun`: the method used to calculate the confidence/credible interval for simulation results. Either “quantile” (default) or “hdi” for highest density region.
- `include_intercept`: logical whether to include the intercept in the pairwise tests (intercept excluded by default).
- `include_zero`: logical whether a univariate null hypothesis test should be included for each estimate (TRUE by default).
- `sig_diffs`: optional logical vector indicating, for each pair of estimates, whether or not there is a significant difference. This is NULL by default. There is a vignette to the package that shows how this can be used.
- `tol`: tolerance level for evaluating the symmetry and positive-definiteness of the variance-covariance matrices (defaults to $1e-08$).
- ... Currently, no other arguments are passed down to other internal functions.

The package has dependencies on six other packages. First, **dplyr** (Wickham et al., 2023) and **tidyr** (Wickham et al., 2024) are used for different data wrangling tasks all throughout the implementation of the procedure. The **ggplot2** package (Wickham, 2016) is used as the main driver of the plot method. **HDInterval** (Meredith and Kruschke, 2022) is used for simulation results where the user specifies the use of highest density region (HDI) to calculate the confidence/credible interval as opposed to a conventional quantile interval. The **multcomp** package (Hothorn et al., 2008) is used to compute adjusted confidence intervals for the original `test_level` tests if they are requested. We import `emmobj()` from **emmeans** (Lenth, 2025) for the auxiliary function, `get_letters()` that helps make compact letter displays. In the examples below, we also use **broom** (Robinson et al., 2025) and **gt** (Iannone et al., 2025) for formatting tabular output.

For the default method, normal theory tests are performed. Specifically, we use R’s `combn()` function to identify all pairwise combinations of observation ids. For example, for a vector of estimates (call it `est` with five values) and a variance-covariance matrix (call it `v`, which is 5×5), we define `combs <- combn(length(est), 2)` which would identify the ten different pairwise combinations. We reorder `est` so they are from largest to smallest and reorder the rows and columns of `v` accordingly, too. We then make a matrix that calculates the pairwise differences in the following way:

```
D <- matrix(0, nrow= length(est), ncol = ncol(combs))
D[cbind(combs[1,], 1:ncol(D))] <- -1
D[cbind(combs[2,], 1:ncol(D))] <- 1
```

This produces a matrix, `D`, such that each column corresponds with a single pairwise test. We can then calculate the pairwise differences and their standard errors with:

```
diffs <- est %*% D
v_diffs <- t(D) %*% v %*% D
se_diffs <- sqrt(diag(v_diffs))
```

²We do not adjust for multiple comparisons in the following examples, but we discuss its impact in a dedicated section of the supplementary material.

Finally, we calculate z-statistics for all pairwise differences and their p-values using the standard normal CDF. If the object inherits the "lm" class and has a `df.residual` element, a t-test is used instead. The vector `s` above, is then defined as $Pr(|z| > Z) < \text{test_level}$ for the vector of z-statistics from the pairwise tests.

Next, we calculate the lower and upper confidence bounds for all test values of γ . This produces a $\text{length}(\text{est}) \times \text{diff}(\text{range_levels}) / \text{level_increment}$ matrix of lower (L) and upper (U) confidence bounds. Since the values of `est` are ordered from largest to smallest, if $L_i > U_j$ then the confidence intervals for stimulus i and j do not overlap for any values of $i < j \leq \text{length}(\text{est})$. Likewise, if $L_i < U_j$, the two intervals do overlap. Thus, each column of the resulting matrix `L[combs[1,],] > U[combs[2,], ]` contains a vector s^* that can be used to evaluate correspondence with s .

For the simulation method, `est` is a $\# \text{ draws} \times \text{length}(\text{est})$ matrix where we calculate pairwise differences of the estimates using the D matrix defined above: `diffs <- est %*% D`. We then calculate the probability that each pairwise difference is greater than zero: `p_diffs <- apply(diffs, 2, \(\x\)\text{mean}(\x > 0))`. Then, to treat credible values greater than zero and credible values smaller than zero the same, we calculate `p_diffs <- ifelse(p_diffs > .5, 1-p_diffs, p_diffs)`. We create the `s` vector as `p_diffs < test_level`. We use `cifun()` at level γ to calculate the lower and upper bounds of each estimate and proceed in the same fashion as above to define s^* and evaluate the correspondence between s and s^* .

Regardless of the method, we identify the level(s) of γ that produce the greatest correspondence between s and s^* . Below, we describe some of the function arguments in greater detail as well as some of the subsequent generic methods (i.e., `print()` and `plot()`) which are defined for objects returned by `viztest()`.

3.1 The `obj` argument

The `obj` argument can be of two different broad types. The default method of `viztest()` uses `coef()` and `vcov()` to extract the estimates and their variance-covariance matrix, respectively. Obviously, this will not work as intended for all analytical situations of interest. To that end, we provide a function `make_vt_data()` that can take a vector of estimates and either a vector of standard errors (in the case of independent estimates) or a variance-covariance matrix in the case of non-independent samples. If a vector of standard errors is provided, a variance-covariance matrix will be produced by multiplying the vector of standard errors by the appropriately sized identity matrix. The final argument of `make_vt_data()` is an argument identifying the type of input data. If `type = "est_var"` the function assumes that `estimates` contains point estimates and variances either a vector of standard errors or a variance-covariance matrix. In this case, the function returns an object of class `vtcustom`. We have defined `coef()` and `vcov()` methods for `.vtcustom` objects so that `viztest()` will work as expected in this case. We anticipate two primary use-cases for this function, though others will likely arise, too.

1. Users may want to replace the default variance-covariance matrix with a robust alternative. In that case, calling `make_vt_data()` on the original estimates and the robust variance-covariance matrix will allow the pairwise tests to be calculated with the robust variance-covariance matrix.
2. It is possible that the user would want to make a plot with a collection of descriptive statistics that do not derive from a single object. In that case, assuming they are independent, the vector of estimates and standard errors could be provided to `make_vt_data()` to evaluate all statistics at the same time.

For simulation results, users can also use the `make_vt_data()` function. In this case, the `estimates` argument is the $\text{draws} \times \text{estimates}$ matrix of simulated values. The `variances` argument remains `NULL` and `type` is set to `"sim"`. This produces an object of class `.vtsim`, that can be passed to `viztest`. Note, that if `type = "sim"`, variances will be disregarded even if they are specified.

3.2 The `sig_diffs` argument

We provide the option for the user to input the vector s directly with the `sig_diffs` argument. The specification of this argument is a bit complex. While it is just a logical vector, care must be taken to ensure that the order of the values is appropriate. The complexity here comes from the fact that, internally, the estimates and variance-covariance matrix are rearranged in decreasing order of the estimates. To aid in the process, we provide a function `make_diff_template()` that takes three arguments – `estimates`, the same estimated values that will be passed to `viztest()`, `include_zero` and `include_intercept`, which should be set to the same values as they are in the `viztest()` function (for convenience, they have the same default values in both places). The function simply reorders the estimates and identifies the pairwise combinations the same way that `viztest()`

does. It returns a data frame that contains the names (taken from `names(estimates)`) of the estimates for the larger and smaller estimate in each pair. The resulting data frame can be merged with estimates or any other values with the same names as the estimates that can help identify the significance or credibility of pairwise differences. If this is provided, it overrides the calculation of the pairwise tests in `viztest()`. There is a vignette that accompanies the R package that discusses how to use `sig_diffs` and `make_diff_template()`.

3.3 viztest() returned values

The `viztest()` function returns a list containing 9 elements:

- `tab`: a table returning the results of the tests for all values of γ . This table is composed of 4 columns:
 - `level`: the tested γ s. The default behaviour is to perform a two-tailed test at this γ level. Use $2 \times \gamma$ to perform a one-tailed test.
 - `psame`: the percentage of agreement between the pairwise test and the visualization test (s_{ij} and s_{ij}^*) for corresponding levels of γ (higher is better).
 - `pdiff`: the percentage of statistically significant differences in pairwise tests (s_{ij}).
 - `easy`: the easiness measure for each γ (higher is better).
- `pw_test`: s_{ij} , a Boolean vector of significance of pairwise tests.
- `ci_tests`: s_{ij}^* , a Boolean vector of significance of visual CI tests.
- `combs`: the matrix describing all the tested pairs.
- `param_names`: the vector of covariates' names in descending order.
- `L`: the matrix of lower bounds for each covariate (rows) across all tested γ (columns).
- `U`: the matrix of upper bounds for each covariate (rows) across all tested γ (columns).
- `est`: a sub-list containing the variable's names (`vb1`), the original estimates (`est`), their standard errors (`se`), and their lower (`lwr_add`) and upper (`upr_add`) confidence bounds at the original `test_level`.
- `call`: an object storing the original call to the function.

The object also has two additional defined attributes: "adjusted", giving the method used for multiplicity adjustment or "none" if none was used and "test_level", giving the type I error rate used for the tests.

Choosing among different γ levels

In the event that there are multiple values of γ that produce identical correspondence between s and s^* , how should we choose among the possibilities? In essence every level "works," but some will make the perceptual tasks required to decode the visualization easier than others. In general, this type of display – inferential confidence intervals plotted along the same axis – is ideal for being able to discriminate even small distances (Cleveland and McGill, 1984). Lu et al. (2022) discuss how people perceive "just noticeable differences" (JNDs) in different kinds of displays – the bar chart being the one most applicable to our situation. They find that the smallest distinguishable distance between two bars grows as the distance between the bars increases. That is, for vertical bars, the farther they are apart on the horizontal axis, the larger the height difference has to be to be visually distinguishable. Given that stimuli have potentially different variances and consequently different size confidence intervals, it is impossible to always place intervals that will be difficult to distinguish from one another in close proximity. This means that we should try to choose γ such that it makes the perceptual task as easy as possible. We do two things to try to make this as easy as possible.

First, we identify the level that will produce plots that require the least cognitive effort to decode. That is, we identify a level that, as much as possible, makes intervals where the relevant overlaps (or non-overlaps) are larger than just noticeable.

To motivate the discussion, we return to the chick weights example above. Here, we are interested in the predictions from the model. We can make those predictions with the `marginalEffects` package (Arel-Bundock et al., 2024). After generating the predictions, we can run `viztest()` on the resulting object. The output below shows that the tests for all pairs of estimates are appropriately captured by the (non-)overlaps of confidence intervals between 68% and 85%.

```
#>
#> Correspondents of PW Tests with CI Tests
#>   level psame pdiff   easy  method
```

```
#> 1  0.68      1  0.8 -13.9378307 Lowest
#> 2  0.76      1  0.8 -2.4617453 Middle
#> 3  0.85      1  0.8 -14.3551587 Highest
#> 4  0.78      1  0.8 -0.8146977 Easiest
#>
#> All 15 tests properly represented for by CI overlaps.
```

Figure 2 plots the boundary intervals (68% and 85%) as well as the midpoint (76%) and what we term the “easiest” level (78%). How easy the display is to engage and how big the differences are that readers will need to distinguish depends on two pairs of estimates in every result. The top row in the figure (labelled “Test 1”) shows the first of these tests – the test for the pair of estimates that are significantly different from each other, but whose confidence intervals are closest to overlapping among all significantly different pairs. In our example below, these two estimates are casein and meatmeal. The 68% interval makes it clearest that there is a difference between these two estimates because the distance between the upper bound for meatmeal and the lower bound for casein is largest. At the other end, using the 85% level it is very difficult to see whether the intervals just overlap or just do not overlap. This difference between the meatmeal upper bound and casein lower bound likely is not noticeable. We want to choose a level that makes significant differences easy to identify. The bottom row of the figure presents the second test. This one is for the two estimates that are not statistically different from each other, but whose confidence intervals are closest to not overlapping among all pairs of estimates that are not statistically different from each other. In our example, these are for meatmeal and soybean. Here, the 85% interval makes it clearest that the two confidence intervals for meatmeal and soybean overlap. Just like the 85% interval did for the first test, the 68% interval makes it nearly impossible to identify whether the intervals for soybean and meatmeal overlap or not. As you can see, picking values at the ends of the range will tend to highlight either the significant differences or the insignificant differences at the expense of the other. The 76% interval makes both tests relatively clear as a value in the middle of the range will tend to do. However, we propose a measure that essentially tries to equalize the size of the overlap for Test 2 and the non-overlap (e.g., the distance between the meatmeal upper bound and the casein lower bound) for Test 1. This is how we define “cognitive easiness.” This is the value that makes the differences as large as possible (hopefully beyond the JND boundary) for the two tests that are most prone to inferential errors. Here, the 78% interval is the one that does that.

The other potential solution is to draw reference lines for ambiguous pairs of estimates – those that are difficult to distinguish. The `plot()` method for `viztest` objects permits this kind of annotation. We turn now to some other use cases that will also allow us to describe the output from the `print()` and `plot()` methods that are defined for the results.

4 Use cases

In this section, we present five use cases where **VizTest** comes in handy. First, an example using a regular regression model. Second, we demonstrate how to use the function with multilevel regression objects. Third, we show how to create a model object compatible with the function such that we may visualize predicted probabilities. Fourth, we similarly show how to create a compatible object out of descriptive quantities such as averages. Finally, we provide a workflow that will allow users to produce forest plots that visualize inferential CIs. The code chunk below loads the different packages used in these examples. **wooldridge** (Shea, 2024), **carData** (Fox et al., 2022) and **rmeta** (Lumley, 2018) are loaded for their datasets; and **dplyr**, **tidyr**, **ggplot2**, **forestplot** (Gordon and Lumley, 2025), **patchwork** (Pedersen, 2024), **lme4** (Bates et al., 2015), **marginaleffects**, and **HDInterval** are loaded for data wrangling and analysis.

```
library(VizTest)
library(wooldridge)
library(carData)
library(rmeta)
library(dplyr)
library(tidyr)
library(ggplot2)
library(forestplot)
library(patchwork)
library(lme4)
library(marginaleffects)
library(HDInterval)
```

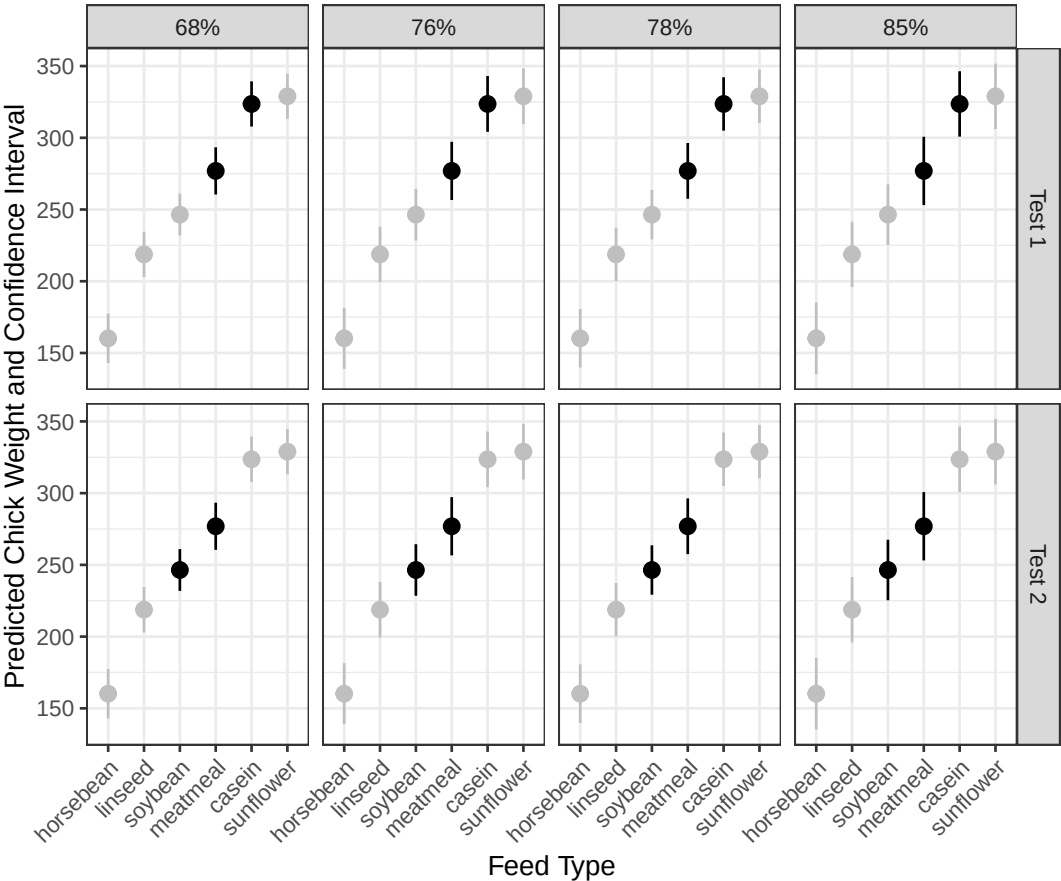



Figure 2: Predicted Chick Weights With Different Confidence Intervals. The display shows predicted chick weights with four different inferential confidence intervals – 68% (the smallest one that works), 85% (the largest one that works), 76% (the middle of the range) and 78% (the one we identify as cognitively easiest). The confidence levels are labelled in the strips at the top of the figure. The two rows refer to two different tests of interest. In the top row, the points that are significantly different, but have the confidence intervals closest to overlapping are plotted in black. The smaller the confidence level, the easier it is to see that there is space between the upper bound for meatmeal and the lower bound for casein. The bottom row considers a converse test – two points that are not significantly different from each other, but whose confidence intervals are closest to not overlapping. In this case, as the confidence level gets larger, it is easier to see the overlap in the confidence intervals and rightly identify the lack of statistical difference. All else equal, values in the middle will tend to strike a balance between getting these two tests right.

In addition to demonstrating how to use the functions, we also demonstrate how you might discuss the results. The notes to the figures could serve as a template for interested users to describe their own results.

4.1 Objects where `coef()` and `vcov()` work

For this example, we use the `gpa1` dataset from the `wooldridge` package. We create a regression model using `lm`. Models created from `lm` and `glm` can directly be used in `viztest` as we do here. In fact, any object that has a defined `coef()` method that returns estimates and a `vcov()` method that returns the variance-covariance matrix of the estimates will work this way. We also make explicit the default values of the `test_level`, `range_levels`, and `level_increment`. The `print()` method for `viztest` objects will print a few different pieces of information. First, it prints a table that identifies the smallest, middle, largest and easiest values of inferential confidence intervals that maximize the correspondence between the pairwise tests and the (non-)overlaps of the confidence intervals. In some cases, this might identify a single value or very small range. In other cases, it could identify quite a large range.

If the `missed_tests` argument is `TRUE` (its default), then the printed results will either indicate that all tests have been accounted for or it will identify which tests are not accommodated by the inferential confidence intervals. The output below indicates that the optimal CI is at the 80% level and that this yields two errors. Which tests are not covered depends on the chosen level. The smallest, middle and easiest levels produce results where the coefficients for `alcohol` and `car` are not significantly different from zero (see the `Insig` designation in the `pw_test` column), but their inferential CIs do not include zero (see the `No` designation in the `ci_olap` column). The highest value in the range (94%) produces different errors. Here, the pairwise tests between `PC` and `male` as well as the test between `alcohol` and `car` are significant, but the 94% intervals still overlap.³ This means that no one inferential CI can visually represent all pairwise test *and* all univariate tests against zero. That said, all the pairwise difference tests are accommodated by the 80% interval.

```
#### 4.1 Regular object ####
model1 <- lm(colGPA ~ skipped + alcohol + PC + male + car + job20, data = gpa1)

# Parsing to viztest
viztestObj <- viztest(
  model1,
  test_level = 0.05,
  range_levels = c(0.25, 0.99),
  level_increment = 0.01,
  include_zero = TRUE
)

# Print
print(viztestObj)

#>
#> Correspondents of PW Tests with CI Tests
#>   level   psame   pdiff   easy method
#> 1  0.73 0.9047619 0.3333333 -0.039414304 Lowest
#> 2  0.81 0.9047619 0.3333333 -0.008306606 Middle
#> 3  0.94 0.9047619 0.3333333 -0.064639897 Highest
#> 4  0.80 0.9047619 0.3333333 -0.002000690 Easiest
#>
#> Missed Tests for Lowest Level (n=2 of 21)
#>   bigger smaller pw_test ci_olap
#> 7 alcohol    zero  Insig     No
#> 15  zero     car   Insig     No
#>
#> Missed Tests for Middle Level (n=2 of 21)
#>   bigger smaller pw_test ci_olap
#> 7 alcohol    zero  Insig     No
#> 15  zero     car   Insig     No
#>
```

³Setting `include_zero=FALSE` shrinks the acceptable range from [73%, 94%] to [73%, 84%] because the univariate tests would not be considered, so the values in the second interval would account for all the pairwise tests.


```
#> Missed Tests for Highest Level (n=2 of 21)
#>      bigger smaller pw_test ci_olap
#> 4      PC      male      Sig      Yes
#> 11 alcohol      car      Sig      Yes
#>
#> Missed Tests for Easiest Level (n=2 of 21)
#>      bigger smaller pw_test ci_olap
#> 7 alcohol      zero Insig      No
#> 15      zero      car Insig      No
```

There are two ways this could be handled. First, we could simply note in the figure caption the two tests that are not appropriately captured by the inferential confidence intervals. Alternatively, if the univariate tests against zero are important to convey, the standard (1-test_level) confidence intervals will appropriately capture those. Those intervals could be plotted as well, though the figure caption should still indicate the discrepancy. The `plot()` method for these objects overlays the inferential and standard confidence intervals by default.

The `plot` method for the object then yields panel (a) of Figure 3. By default, it plots the estimates and their optimal CI using the `geom_point()` and `geom_linerange()` function from `ggplot2`, as well as the (1-test_level) confidence interval. The original `test_level` can be removed from the plot with `add_test_level=FALSE`. In addition to the `viztest` object, the method takes several arguments:

- `add_test_level`: logical indicating whether the (1-test_level) confidence intervals should also be plotted (if returned from the `viztest()` function). Default is `TRUE`.
- `ref_lines`: one of `all`, `ambiguous`, `none`, or a vector of coefficient names (they must match the names included in the `viztest` object). `all` will plot vertical dotted lines along the upper bound of the lowest coefficient to the most distant coefficient with overlapping confidence intervals. `ambiguous` will draw these lines but only for the overlapping bounds that are near to each other. `none`, the default value, won't draw reference lines. There is also the option of manually feeding the argument a vector of coefficient names for which the user wants reference lines drawn.
- `viz_diff_thresh`: threshold value for identifying ambiguity that will be used if `ref_lines = "ambiguous"`. Default at 0.02.
- `make_plot`: a Boolean value indicating whether to directly plot the results of the object (`TRUE`, the default) or to return a dataframe that can be used to make a custom visualization.
- `level`: allows users to choose the CI level they want, either through a numerical value or one of the following: `ce` for cognitively easiest, `max` for highest, `min` for smallest, and `median` for the middle value. The default is to return the cognitively easiest.
- `trans`: a function that transforms the coefficients and their CI. Default is the identity function `I`, i.e. no transformation. Note, non-linear transformations do not trigger a recalculation of the test statistics, they simply result in a transformation of the estimate and the ends of the confidence bounds.
- `est_point_args`: A list of arguments to be passed to `geom_point()` that plots the point estimates. The default is `list(color="black", size=2)`.
- `opt_ci_args`: A list of arguments to be passed to `geom_linerange()` to plot the optimal visual testing intervals. The default is a blank list.
- `test_ci_args`: A list of arguments to be passed to `geom_linerange()` to plot the (1-test level) confidence intervals. The default is a blank list.
- `ref_line_args`: A list of arguments to be passed to `geom_segment()` to plot the reference lines. The default is `list(color="gray75", linetype=3)`.
- `scale_linewidth_args`: A list of arguments to be passed to `scale_linewidth_manual()` to change the thickness of the confidence intervals. The default is `list(values=c(3.5, .5))`.
- `scale_color_args`: A list of arguments to be passed to `scale_color_manual()` to change the default colour of the different confidence intervals when `add_test_level=TRUE`. The default is `list(values = c("gray75", "black"))`.
- `overall_theme`: A theme function that will be passed to the `ggplot` call before `theme()`. Default is `theme_bw`.
- `theme_arg`: A list of arguments to be passed to `theme()` to modify the theme of the plot. The default is `list(legend.position="top")`.
- `remove_caption`: A caption is printed by default that alerts the user to the adjustment of the confidence intervals for visual testing. If `remove_caption=TRUE`, the caption will be removed from the plot.

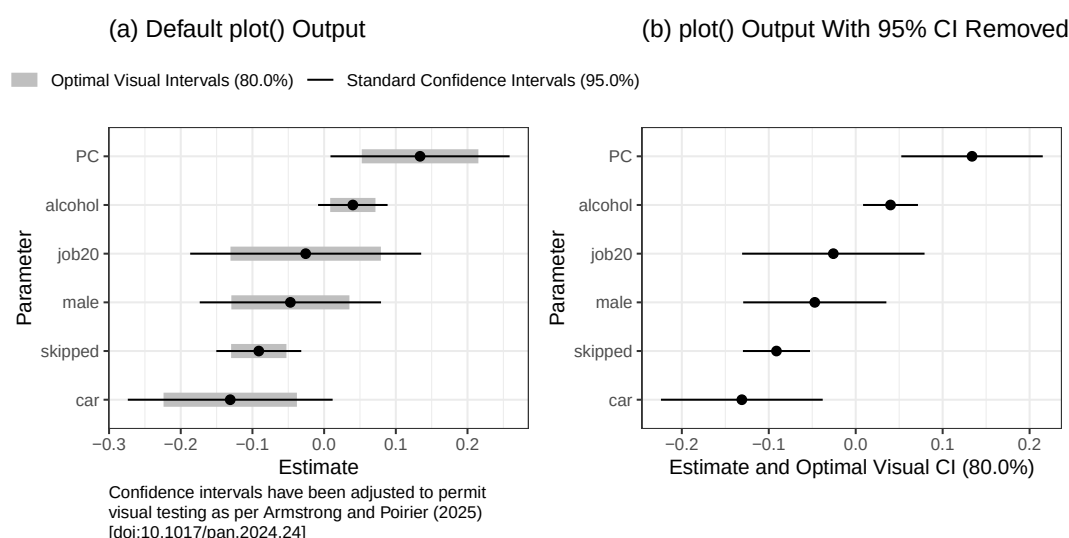


Figure 3: Regression Object Plot With Inferential and Standard Confidence Intervals. The figure above presents coefficient estimates from an OLS regression estimated with `lm()` in R. It also includes standard 95% confidence intervals (thin black line) and inferential confidence intervals (thick gray bar) that present visual testing. For any pair of estimates, if the inferential confidence intervals overlap, the estimates are not statistically different from each other at the 5% level. If they do not overlap, the estimates are statistically different from each other at the 5% level. Note that the inferential confidence intervals for alcohol and car do not include zero, but the estimates are not statistically different from zero at the 5% level. The standard confidence intervals capture tests against zero appropriately.

```
p1 <- plot(viztestObj) +
  labs(title = "(a) Default plot() Output")
p2 <- plot(viztestObj,
  add_test_level = F,
  remove_caption = T) +
  labs(title = "(b) plot() Output With 95% CI Removed")

p1 | p2
```

There are a few different ways that we could account for tests not accommodated by the inferential confidence intervals. We could simply discuss them in the figure's note. In this case, we would simply add a note to the effect: "Even though the alcohol inferential confidence interval does not include zero, it is not statistically different from zero in a proper pairwise test." In this particular case (or any case where tests against zero are not captured), the 95% confidence intervals will capture those tests (since they are univariate tests against a point null hypothesis). Hence, by simply adding the 95% confidence intervals to the plot, we solve this specific case's missed tests. In situations where 95% confidence intervals aren't necessary and might obstruct a more complex visualization exercise, setting `add_test_level=FALSE` will remove them as shown in panel (b) of Figure 3.

Replacing or adding to default confidence intervals

In [Armstrong II and Poirier \(2025\)](#), we follow the convention in the literature and suggest replacing the default (e.g., 95%) confidence intervals with the ones that best accomplish the visual testing goal. This is the reasoning that many are using for presenting 84% intervals – not because they are inherently interested in 84% confidence, but because they facilitate pairwise tests at the 5% level.

We recognize that for many, trying to publish a paper with, say, 72% confidence intervals and convincing reviewers that they are not doing something disingenuous will be a heavy lift. In these cases, there is no harm in also presenting the 95% intervals. The only real cost is increased cognitive load required to decode the graph owing to the two different sets of intervals. If they are well explained, they should provide sufficient information to both allay fears of reviewers and provide readers with the ability to perform any arbitrary set of pairwise tests. Hence, the present version of the package displays the original test's intervals by default.

4.2 Multilevel regression objects

For this example we use the World Values Survey dataset from [carData](#) and produce a multilevel regression model using `lmer()` from the [lme4](#) package.

```
#### 4.2 Multilevel regression objects ####
data(WVS, package = 'carData')

# Poverty variable as a scale from -1 to 1
NewWVS <- WVS %>%
  mutate(povertyNum = as.numeric(poverty) - 2)

# The model
model2 <- lmer(povertyNum ~ age + religion +
  degree + gender + (1 | country), NewWVS)
```

Here, the results have class `lmerMod` which is not readily usable by `viztest`. For `lmerMod` and `merMod` objects, `coef()` returns a matrix of coefficients with level-2 observations in the rows and variables in the columns. Thus, `coef()` is defined, but produces output that is not compatible with `viztest()`. To proceed, we need to create a custom object; we can do this with `make_vt_data()`. First, we extract the fixed effects from the model and assign them cleaner names for the subsequent plot. Second, we extract the variance-covariance matrix using `vcov` which in the case of `lmerMod` objects outputs an object of class `dpoMatrix`. The `make_vt_data()` function does a test of whether the input variances inherit the “matrix” class. This fails for `dpoMatrix` objects, so we recast the object as a matrix using `as.matrix`. Now that we have both a named vector of coefficients and its corresponding variance-covariance matrix, we create a compatible object fed to `viztest()`.

```
# Creating custom object
## Extracting fixed effect coefficients and naming them
named_coef_vec <- fixef(model2)
names(named_coef_vec) <- c("(Intercept)", "Age", "Religious", "Has a degree", "Male")

## Extracting vcov matrix
#### As matrix necessary to overwrite special object from lme4
vcov <- as.matrix(vcov(model2))

eff_vt <- make_vt_data(named_coef_vec, vcov)

# Parsing to viztest
viztestObj <- viztest(
  eff_vt,
  test_level = 0.05,
  range_levels = c(0.25, 0.99),
  level_increment = 0.01
)

# Print
print(viztestObj)

#>
#> Correspondents of PW Tests with CI Tests
#>   level psame pdiff      easy method
#> 1  0.95     1   0.5 -0.008101835 Lowest
#> 2  0.96     1   0.5 -0.003928933 Middle
#> 3  0.98     1   0.5 -0.009654884 Highest
#> 4  0.97     1   0.5 -0.001339149 Easiest
#>
#> All 10 tests properly represented for by CI overlaps.
```

Any inferential confidence level from 0.46 to 0.90 properly represents all pairwise tests. We then plot the estimates in Figure 4, specifying that we want to display the cognitively easiest level with all reference lines.

```
# Plotting
```

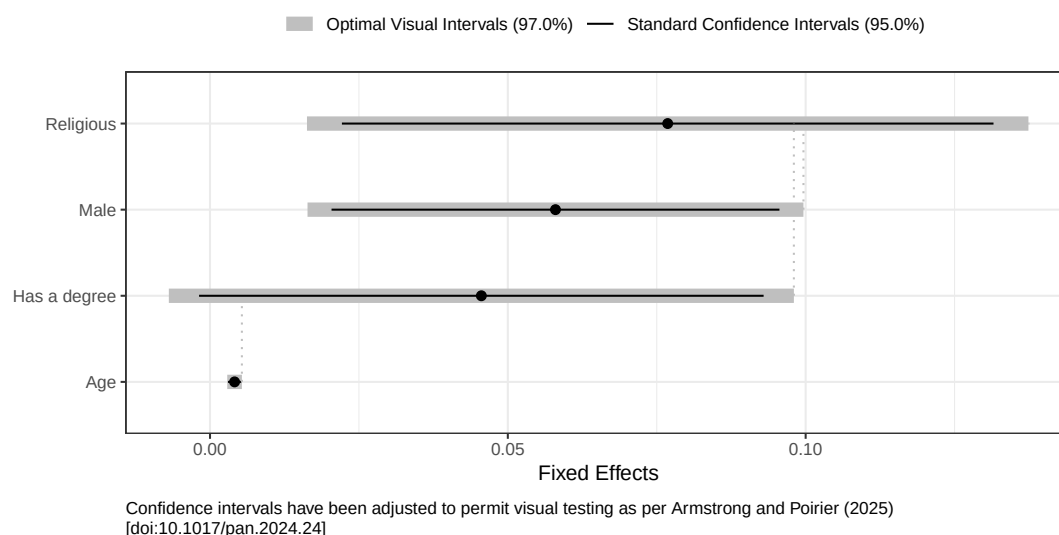


Figure 4: Multilevel Regression Objects. The figure above presents the default output of the plot method when all reference lines are included. Displayed are the standard 95% confidence intervals (thin black line), inferential confidence intervals (thick gray bar) that present visual testing, as well as dotted lines that connect the upper bound of the inferential CIs with all overlapping bounds. As expressed in the printed output above, all tests including ones against 0 are appropriately represented.

```
plot(viztestObj, ref_lines = "all", level = "ce") +
  labs(y = "", x = "Fixed Effects")
```

4.3 Predicted probabilities

For this example, we use the TitanicSurvival dataset from [carData](#) to create predicted probabilities of survival based on a logistic regression model. We first create a variable corresponding to age categories and then run a model including an interaction between sex and this new categorical age variable. We use the `avg_predictions()` function from [marginaleffects](#) to produce predicted values for each pair of values of the interaction. One of the benefits of using the functions in [marginaleffects](#) is that they have defined `coef()` and `vcov()` methods which make them compatible with `viztest()`. However, one thing to note is that by default, the standard errors are calculated with the delta method and the confidence intervals are normal theory intervals around the estimated values. For predicted probabilities close to zero or one, users may want to use a different method. We show both below.

Normal theory intervals

Since the default behaviour for `avg_predictions()` is to use normal theory intervals, we do not need to intervene in the function. First, we can manage the data, estimate the model and then calculate the average predicted probabilities.

```
#### 4.3 Predicted probabilities ####
data(TitanicSurvival, package = "carData")
NewTitanicSurvival <- TitanicSurvival %>%
  mutate(
    ageCat = case_when(
      age <= 10 ~ "0-10",
      age > 10 & age <= 18 ~ "11-18",
      age > 18 & age <= 30 ~ "19-30",
      age > 30 & age <= 40 ~ "31-40",
      age > 40 & age <= 50 ~ "41-50",
      age > 50 ~ "51+"
    )
  )
# The model
model3 <- glm(
```

```

survived ~ sex * ageCat + passengerClass,
data = NewTitanicSurvival,
family = binomial(link = "logit")
)

# Predicted values
mes <- avg_predictions(model3, variables = list(
  ageCat = levels(NewTitanicSurvival$ageCat),
  sex = levels(NewTitanicSurvival$sex)
))

```

Next, we can find the appropriate inferential confidence levels. In this case, since we are considering predicted probabilities, we can exclude zero as the univariate test against zero is not particularly interesting. All 66 pairwise tests are accounted for by the (non-)overlaps in the inferential confidence intervals between 84% and 95%, with 91% being the easiest which we display in Figure 5.⁴

```

#>
#> Correspondents of PW Tests with CI Tests
#>   level psame   pdiff      easy method
#> 1  0.84     1 0.5757576 -0.03453028 Lowest
#> 2  0.89     1 0.5757576 -0.01123875 Middle
#> 3  0.95     1 0.5757576 -0.03239279 Highest
#> 4  0.91     1 0.5757576 -0.00048465 Easiest
#>
#> All 66 tests properly represented for by CI overlaps.

```

One slight inconvenience of using the `marginalEffects` functions is that when multiple variables are involved in the prediction, the names of the estimates are not intuitive. So, instead of using the plot method for our visual testing result, we can simply make the appropriate confidence interval in the existing average prediction data.

```

mes <- mes %>%
  mutate(lwr91 = estimate - qnorm(.91) * std.error,
         upr91 = estimate + qnorm(.91) * std.error)

ggplot(mes,
  aes(
    y = ageCat,
    x = estimate,
    xmin = lwr91,
    xmax = upr91,
    colour = sex
  )) +
  geom_pointrange(position = position_dodge(width = .5)) +
  scale_colour_manual("Sex", values = c("gray50", "black")) +
  theme_bw() +
  theme(legend.position = "top") +
  labs(x = "Predicted Pr(Survival)\nInferential Confidence Intervals (91%)",
       y = "Age Category")

```

Using simulation method

The `marginalEffects` functions have an inference engine that will treat the input models as Bayesian (assuming ignorance priors) and sample from the multivariate normal distribution centred at the coefficient estimates with a variance-covariance matrix equal to the analytical variance-covariance matrix from the model. We can invoke this with the `inferences()` function.⁵

⁴In an actual paper, it would make more sense to use the 95% as it captures all the tests correctly. We display the 91% for exposition here.

⁵We direct the interested reader to Arel-Bundock (2026), chapter 14, section 3 where the author of the `marginalEffects` package discusses the procedure behind the quasi-Bayesian simulations produced by the `inferences()` function. We also provide advice on simulation length in the supplementary material.

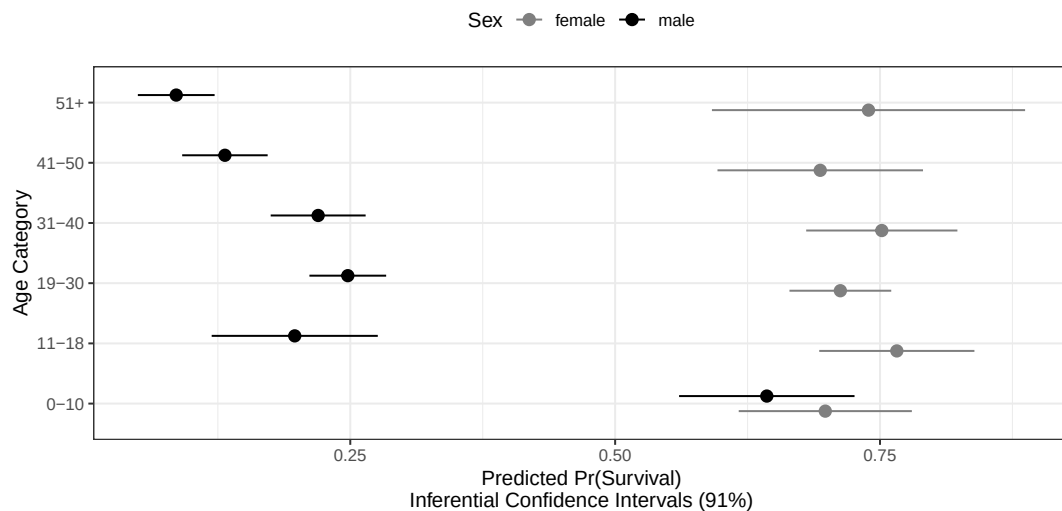


Figure 5: Average Predicted Probabilities. The figure above is generated manually using the raw output from `avg_predictions()`. By passing this output to `viztest()`, we identify the appropriate inferential confidence level, and manually compute the corresponding inferential CIs.

```
ap_sim <- avg_predictions(model3, variables = list(
  ageCat = levels(NewTitanicSurvival$ageCat),
  sex = levels(NewTitanicSurvival$sex)
)) %>%
  inferences(method = "simulation", R = 2500)
```

The “posterior” attribute of the resulting object has the estimates-by-draws posterior simulation values. We transpose that matrix and add some column names. We then pass that to `make_vt_data()` with the argument `type="sim"` and call `viztest()`. Inferential highest density intervals between 84% and 95% all account for the pairwise tests perfectly. The 91% HDIs are easiest to evaluate.

```
post <- t(attr(ap_sim, "posterior"))
colnames(post) <- paste0("b", 1:ncol(post))
post_vt <- make_vt_data(post, type="sim")
vt_sim <- viztest(post_vt, cifun="hdi", include_zero=FALSE)
vt_sim
```

```
#>
#> Correspondents of PW Tests with CI Tests
#> level psame pdiff easy method
#> 1 0.84 1 0.5757576 -0.038920305 Lowest
#> 2 0.89 1 0.5757576 -0.016153394 Middle
#> 3 0.95 1 0.5757576 -0.036469956 Highest
#> 4 0.91 1 0.5757576 -0.003455154 Easiest
#>
#> All 66 tests properly represented for by CI overlaps.
```

While the `plot()` method works with simulation results as well, it is actually a bit easier to simply attach the appropriate credible intervals to the output from `avg_predictions()`.

```
hdis <- apply(post, 2, \x)hdi(x, 0.91))
mes <- mes %>%
  arrange(ageCat) %>%
  mutate(lwr91 = hdis[1, ], upr91 = hdis[2, ])

ggplot(mes,
  aes(
    y = ageCat,
    x = estimate,
    xmin = lwr91,
    xmax = upr91,
```

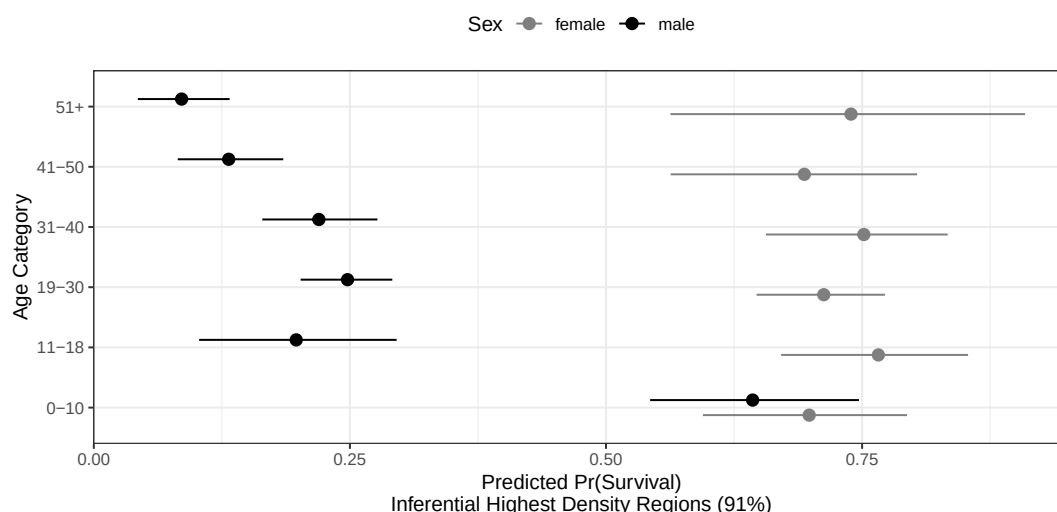



Figure 6: Average Predicted Probabilities. The figure above is generated manually using the raw output from the simulation results. By first calling `viztest` on the custom simulation object, we identify the appropriate inferential confidence level, which we use to produce inferential HDIs.

```

colour = sex
)) +
geom_pointrange(position = position_dodge(width = .5)) +
scale_colour_manual("Sex", values = c("gray50", "black")) +
theme_bw() +
theme(legend.position = "top") +
labs(x = "Predicted Pr(Survival)\nInferential Highest Density Regions (91%)",
y = "Age Category")

```

4.4 Descriptive quantities

For this example, we use the CES11 dataset from [carData](#), calculating the average importance given to religion per Canadian province in 2011. First, we create a numerical scale of the importance of religion from 0 to 1 and calculate the mean and sampling variance for each province.

```

#### 4.4 Descriptive quantities ####
data(CES11, package = "carData")
NewCES11 <- CES11 %>%
  mutate(rel_imp = (as.numeric(importance) - 1) / 3) %>%
  group_by(province) %>%
  summarise(mean = mean(rel_imp), samp_var = var(rel_imp) / n())

```

We then create a named vector of the averages per province and pass that vector along with the vector of sampling variances to `make_vt_data()`.

```

# Creating a vtcustom object
## Vector for the means
means <- NewCES11$mean
names(means) <- NewCES11$province

vt_ces_data <- make_vt_data(means, NewCES11$samp_var)

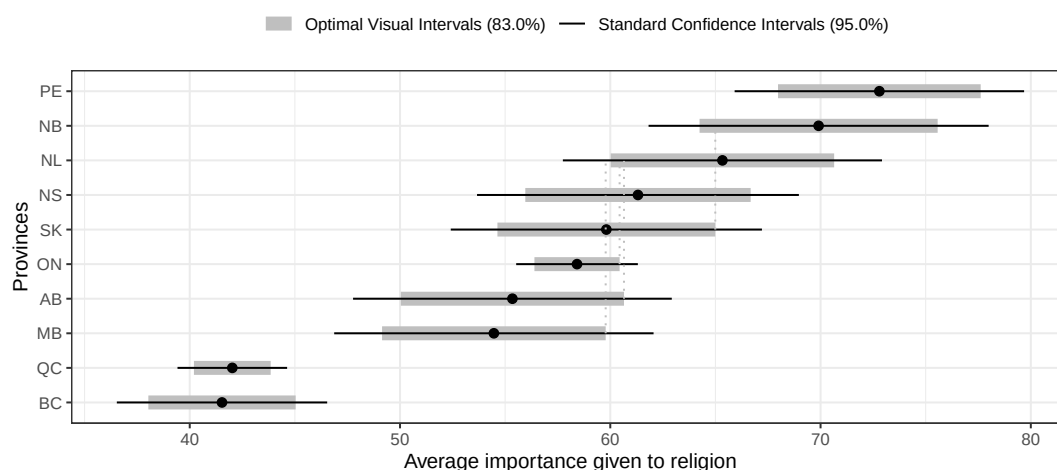
```

Passing the newly formed object to `viztest` indicates that any inferential confidence level from 0.81 to 0.83 properly represents all pairwise tests.

```

# Passing to viztest
viztestCES <- viztest(
  vt_ces_data,
  test_level = 0.05,

```



Confidence intervals have been adjusted to permit visual testing as per Armstrong and Poirier (2025) [doi:10.1017/pan.2024.24]

Figure 7: Descriptive Quantities. The figure above presents the average importance given to religion by each Canadian province. It was produced by first creating a custom object with `make_vt_data()` (feeding it the means and variances for each province) and passing it on to `viztest()`. We can then employ the `plot` method with two modifications. First, using the `trans` argument to transform the raw means (varying from 0 to 1), multiplying them by 100. Then, adding reference lines (dotted gray lines) for ambiguous CIs. Ones where it is difficult to tell whether the bounds overlap or not, that is.

```
range_levels = c(0.25, 0.99),
level_increment = 0.01,
include_zero = FALSE
)

# Print
viztestCES

#>
#> Correspondents of PW Tests with CI Tests
#>   level psame   pdiff      easy method
#> 1  0.81     1 0.555556 -0.006229373 Lowest
#> 2  0.82     1 0.555556 -0.002278028 Middle
#> 3  0.83     1 0.555556 -0.001839989 Highest
#> 4  0.83     1 0.555556 -0.001839989 Easiest
#>
#> All 45 tests properly represented for by CI overlaps.
```

We then plot the estimates in Figure 7, where this time we create a function that will multiply the estimates and their confidence bounds by 100. This results in a display of the importance of religion per province on a scale from 0 to 100 instead of 0 to 1.

```
# plotting
plot(viztestCES, trans = \(x)x * 100, ref_lines = "ambiguous") +
  labs(y = "Provinces", x = "Average importance given to religion")
```

4.5 Forest plot

Forest plots are more common in medical studies than in the social sciences. They are often used for visualizing the results of a meta-analysis, though they could be used for any set of coefficient estimates, too. While the many forest plotting functions in R are quite general, they tend not to anticipate our particular use case – printing two confidence intervals for a single estimate. Functions like `forestplot()` from the `forestplot` package (Gordon and Lumley, 2025) and `forest()` from the `forestploter` package can be used in this case, but adding another confidence interval is idiosyncratic and hacky. It makes the endeavour not particularly user-friendly. To alleviate some of these problems, we have created a suite of `geom_` functions that work with `ggplot2` to make forest plots. We describe the functions here, but also detail them more fully in the package vignette on creating forest plots.

For data, we use the catheter data (Veenstra et al., 1999) from the `rmeta` package (Lumley, 2018). We discuss the workflow for creating the plot below.

First, we need to load the data and we can use the `meta.MH()` function from the `rmeta` package to estimate the Mantel-Haenszel log-odds ratios for each study along with its standard error.

```
library(rmeta)
data(catheter)

## Remove NA observations for the outcome variables
catheter <- catheter[complete.cases(catheter[, c("col.trt", "col.ctrl")]), ]

## Estimate the MH model for the data
cath <- meta.MH(n.trt,
               n.ctrl,
               col.trt,
               col.ctrl,
               names = Name,
               data = catheter)

cath

#> Fixed effects ( Mantel-Haenszel ) Meta-Analysis
#> Call: meta.MH(ntrt = n.trt, nctrl = n.ctrl, ptrt = col.trt, pctrl = col.ctrl,
#>   names = Name, data = catheter)
#> Mantel-Haenszel OR =0.44    95% CI ( 0.36, 0.54 )
#> Test for heterogeneity: X^2( 11 ) = 25.37 ( p-value 0.008 )

names(cath)

#> [1] "logOR"      "selogOR"     "logMH"       "selogMH"     "MHtest"
#> [6] "het"        "call"        "names"       "conf.level"  "statistic"
```

As you can see from above, the `logOR` are the log-odds ratios and `selogOR` are the standard errors of the log-odds ratios. We can use these as the inputs to `make_vt_data()`. There are some `-Inf` values in the log-odds ratios, so we will remove them before creating the data for the `viztest()` function.

```
cath_b <- cath$logOR
names(cath_b) <- cath$names
cath_v <- cath$selogOR^2
if (any(!is.finite(cath_b))) {
  i <- which(!is.finite(cath_b))
  cath_b <- cath_b[-i]
  cath_v <- cath_v[-i]
}
```

We can make the data for `viztest()` and then execute the `viztest()` function on the data. Since the data are in log-odds ratios, normal theory intervals and pairwise tests should make sense. Below, we see that (non-)overlaps in the 86% confidence interval will capture all 66 tests.

```
cath_vt_data <- make_vt_data(cath_b, cath_v)
cath_vt <- viztest(cath_vt_data, include_zero = FALSE)
cath_vt

#>
#> Correspondents of PW Tests with CI Tests
#> level psame pdiff easy method
#> 1 0.86 1 0.2878788 -0.02396481 Lowest
#> 2 0.86 1 0.2878788 -0.02396481 Middle
#> 3 0.86 1 0.2878788 -0.02396481 Highest
#> 4 0.86 1 0.2878788 -0.02396481 Easiest
#>
#> All 66 tests properly represented for by CI overlaps.
```

Probably the easiest way to retrieve the relevant data would be to call `plot(cath_vt, make_plot=FALSE)`, to get the plotting data returned. We could then exponentiate the estimates and confidence bounds to put them on the odds ratio scale. It is worth noting that from the individual studies, there is no reason to believe that the studies would not be independent, and thus the results of the studies are uncorrelated. In that case, building a diagonal covariance matrix from standard errors makes sense. As for the summary measure, it will be correlated with all studies at some level, so we exclude the summary measure from the input to `viztest()`. We add the summary row back in below.

```
plot_dat <- plot(cath_vt, viz_diff_thresh = .01, make_plot = FALSE)
plot_dat <- bind_rows(
  plot_dat,
  data.frame(
    vbl = "Summary",
    est = cath$logMH,
    se = cath$selogMH,
    lwr = NA,
    upr = NA,
    lwr_add = cath$logMH - 1.96 * cath$selogMH,
    upr_add = cath$logMH + 1.96 * cath$selogMH
  )
)
plot_dat <- plot_dat %>%
  mutate(
    mean = exp(est),
    lower = exp(lwr),
    upper = exp(upr),
    lower_add = exp(lwr_add),
    upper_add = exp(upr_add),
    or = sprintf("%.2f", mean)
  )
```

We can join the plotting data with the original data to make sure we have all the relevant data together.

```
cath_plot <- plot_dat %>%
  dplyr::select(vbl, mean, se, lower, upper, lower_add, upper_add, or) %>%
  left_join(catheter, by = join_by(vbl == Name))
```

The functions we wrote require a few additional variables that are easy to make. The `is_summary` variable is a logical vector indicating which (if any) rows represent the summary. We also need a variable indicating the variable used to scale the point sizes. We're using the inverse of the variance in this case. Finally, we will need a variable that is ordered the right way to put summary rows at the bottom (the smallest value is plotted at the bottom). There is a utility in our package called `reorder_forest` that will reorder the values of some label based on the values of another variable.

```
cath_plot <- cath_plot %>%
  mutate(
    is_summary = vbl == "Summary",
    pt_size = 1 / se^2,
    row = reorder_forest(vbl, mean)
  )
```

We use the `gg_forest()` function from the [VizTest](#) package to make the forest plot with both standard and inferential confidence intervals. The `gg_forest()` function uses several functions from `VizTest` that were designed to help build forest plots – `geom_forestpoint()`, `geom_foreststripe()`, `geom_foresttable()` and `scale_x_foresttable()`. These can be used to build the individual table and figure components of the forest plot for more customizable look. The `gg_forest()` function is a wrapper that makes constructing the plots a bit easier. The `y` variable should be ordered in the way the values on the `y`-axis should appear. The `x` variable should be the estimate. The `xmin_std` and `xmax_std` variables are the standard confidence intervals and `xmin_ici` and `xmax_ici` are the inferential confidence intervals. The `size_prop` variable is used to scale the point sizes. The `is_summary` variable is used to indicate which rows are summary rows, so they can be plotted differently. The `use_log_scale` argument indicates whether the `x`-axis should be on the log scale. The `data_cols` argument is a named vector indicating which variables in the data should be used for the table columns and what their labels

should be. The `max_size` argument indicates the maximum size of the points; this may take some trial and error to get just right. The `table_header_size` and `table_text_size` arguments indicate the size of the table header and text, respectively. The `col_nudge` argument indicates how much to nudge the table columns to the left or right. This is particularly useful for left-aligned columns. In that case, the left edge of the text will appear aligned with the tick mark, but the header label will be centred on the tick mark. As such, left-aligned columns will often have to be moved to the left slightly. Finally, the `diamond_aspect` and `diamond_row_frac` arguments indicate how to plot the summary diamond – its aspect ratio and the maximum amount of the row it should take up, respectively. The object, of class `gg_forest` contains two elements – a table and figure, both `ggplot` objects. The plotting method for the object uses the [patchwork](#) package to combine the two into a single plot.

```
out <- gg_forest(
  cath_plot,
  y = "row",
  x = "mean",
  xmin_std = "lower_add",
  xmax_std = "upper_add",
  xmin_ici = "lower",
  xmax_ici = "upper",
  size_prop = "pt_size",
  is_summary = "is_summary",
  use_log_scale = TRUE,
  data_cols = c(
    "Study" = "vbl",
    "Bacteria\n(Coated)" = "col.trt",
    "Bacteria\n(Standard)" = "col.ctrl",
    "OR" = "or"
  ),
  max_size = 8,
  vline = 1,
  table_header_size = 13,
  table_text_size = 5,
  col_nudge = c(-.2, 0, 0, 0),
  diamond_aspect = 25,
  diamond_row_frac = .9
)
plot(out)
```

5 Computational performance and efficiency

There are a couple of aspects of computational performance and efficiency that bear consideration. First is, what is the computational load of `viztest()`? The answers are different depending on the context.

When using normal theory tests, the models need not be re-estimated, so the initial sample size and model complexity are of no real importance. The function simply uses the estimates and variance-covariance matrix to perform the relevant tests and make the relevant confidence intervals. What does matter to some degree is the number of estimates. For m estimates, there are $\frac{m(m-1)}{2}$ pairs of estimates that require evaluation. We also imagine that our approach will fail in high dimensions. By failure we don't mean that it will not "work," we mean that the display that would be generated would be too difficult to decode to be of much use. We don't know the exact number and that may depend on context, but 100 estimates seems like a reasonable first guess. If we think of this as the boundary size of inputs to our analysis, we can give some descriptive evidence about computation times. For reference, our computer is an Apple M2 Max with 96 GB of RAM using macOS Sequoia 15.6.1.

With 100 estimates the average time to run `viztest()` with the default grid without a multiplicity adjustment is a bit less than one second. There is considerable overhead when using `glht()` to calculate the confidence intervals with multiplicity adjustments, so the average run time increases to around seven seconds for the Holm adjustment and around 9 seconds for the Benjamini and Hochberg adjustment. So, even in these cases, computation time is not particularly problematic. This operation on 100 estimates used about 228 MB of RAM.

The simulation results are where the real concern lies. The reason is that the algorithm does not calculate the means and variance-covariance matrix from the simulation and use normal theory to do

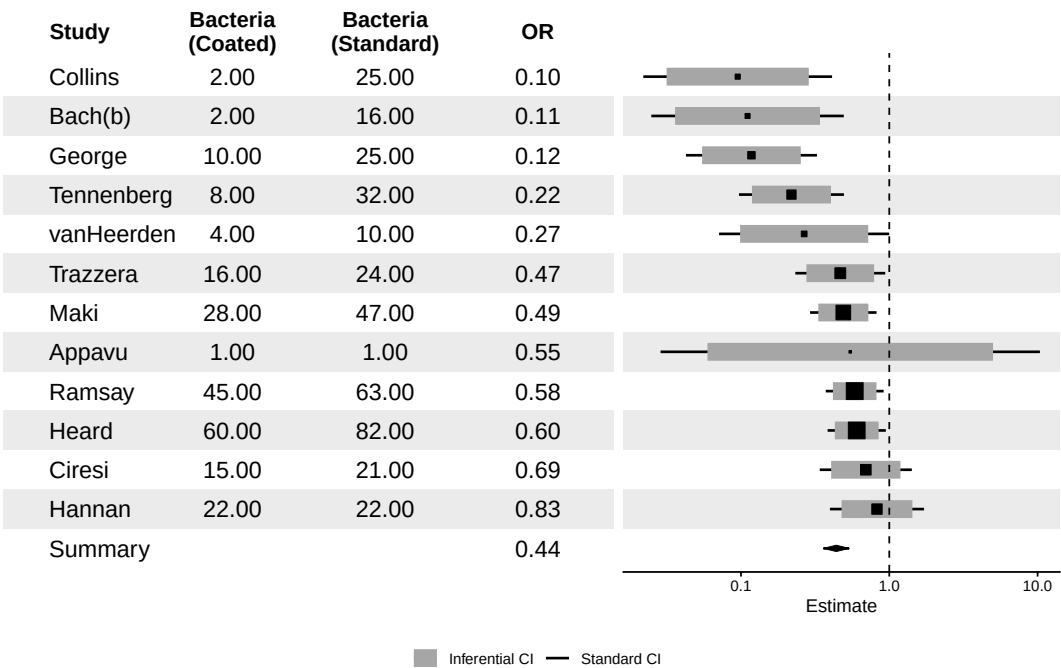


Figure 8: Forest Plot With Both Standard (95%) and Optimal Visual Testing (86%) Confidence Intervals for the Catheter Data

the tests, it uses the full simulation distribution. This means that both the number of estimates and the number of simulations matter. Figure 9 shows the memory usage and elapsed time under different simulation loads. For 100 estimates and 10,000 simulation draws, the viztest() on the default grid takes around 6 seconds and uses around 1.1GB of RAM. The time required by the algorithm and the memory usage scale about linearly with the number of simulation draws (time=25 seconds, RAM = 5.5GB for 50,000 draws, time=50 seconds, RAM = 10.9GB for 100,000 draws). We used gc() to calculate the maximum memory used before and after running the function and took the difference to get these figures. This memory usage does not include the overhead needed to store the simulation object itself. The RAM usage seems more problematic than the time – the algorithm still completes in under a minute even with 100 estimates and 100,000 draws.

If the simulation length is problematic, users could consider calculating the means and the variance-covariance matrix from the simulation and use the normal theory tests as an approximation. This will likely not be perfect, but there are many times where regression results or predictions could be transformed into something approaching normality. For example, predicted probabilities could be subjected to the log-odds transformation and then the means and variances calculated on that scale would be well suited to normal theory tests. The results could then be transformed back into the probability scale if desired.

Another aspect of computational efficiency worth considering is the resolution of the grid. The default grid has 75 values in increments of 0.01 between .25 and .99. This is likely sufficient for most applications. However, there may be reasons to consider either a finer or coarser grid. A coarser grid will decrease computation time, but may not find the optimal value that perfectly corresponds with the pairwise tests even if it exists. A finer grid will simply take longer to run, though it will be more likely to arrive at an optimal solution. Considering our World Values Survey Post-materialism example from above, we could see how this works out. We will use an unadjusted analysis and one with the Holm adjustment because they show slightly different things.

Figure 10 shows the proportion of pairwise tests captured by visual testing intervals using a low-resolution grid (15 evaluation points). The left panel shows the unadjusted result and suggests zooming in between .8 and .9. The right panel shows the Holm adjustment and suggests extending the range to higher values and starting much closer to 1, say around .95. We could use this information to target those areas with a higher resolution. Figure 11 shows the proportion of pairwise tests captured by the visual testing levels, but using a higher-resolution grid that is more targeted along the x-axis. The optimal values are clearly identifiable in this figure. This general idea could be used by others to maximize computational efficiency.

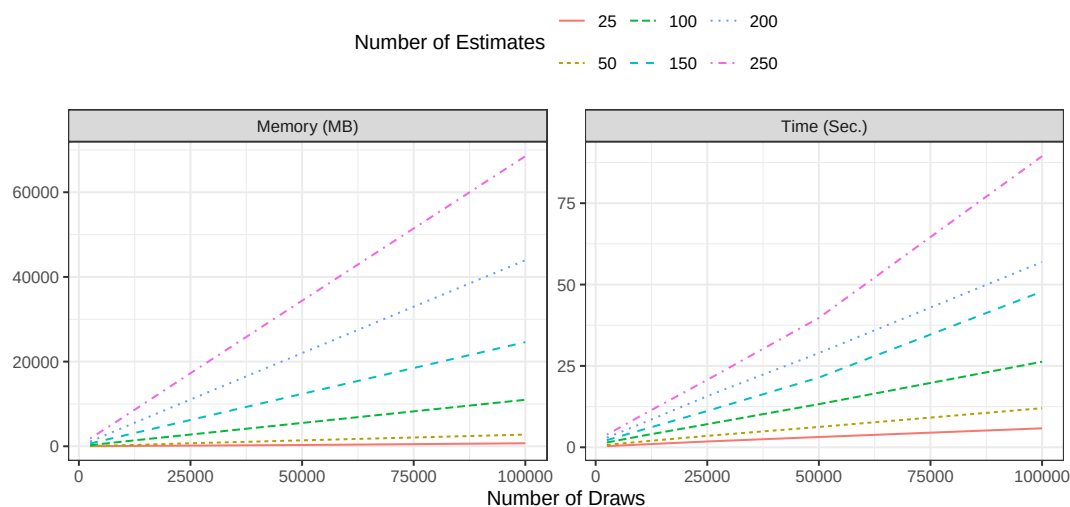


Figure 9: Computation Time and RAM Usage by Number of Estimates and Number of Draws for Simulation Studies

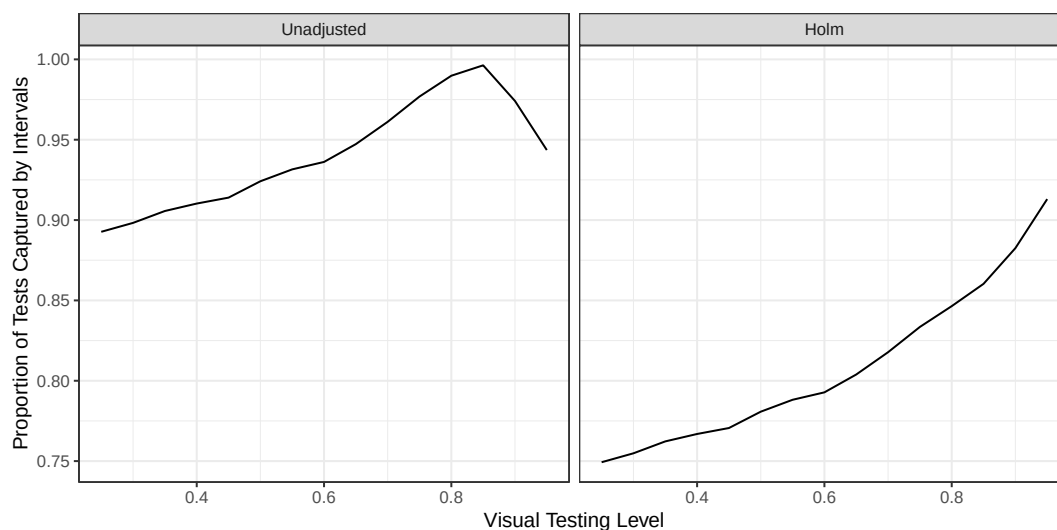


Figure 10: Proportion of Pairwise Tests Captured by Visual Testing Intervals Using a Low Resolution Grid. The grid uses 15 evaluation points. The left panel shows the unadjusted result and suggests zooming in between .8 and .9. The right panel shows the Holm adjustment and suggests extending the range to higher values and starting much closer to 1, say around .95.

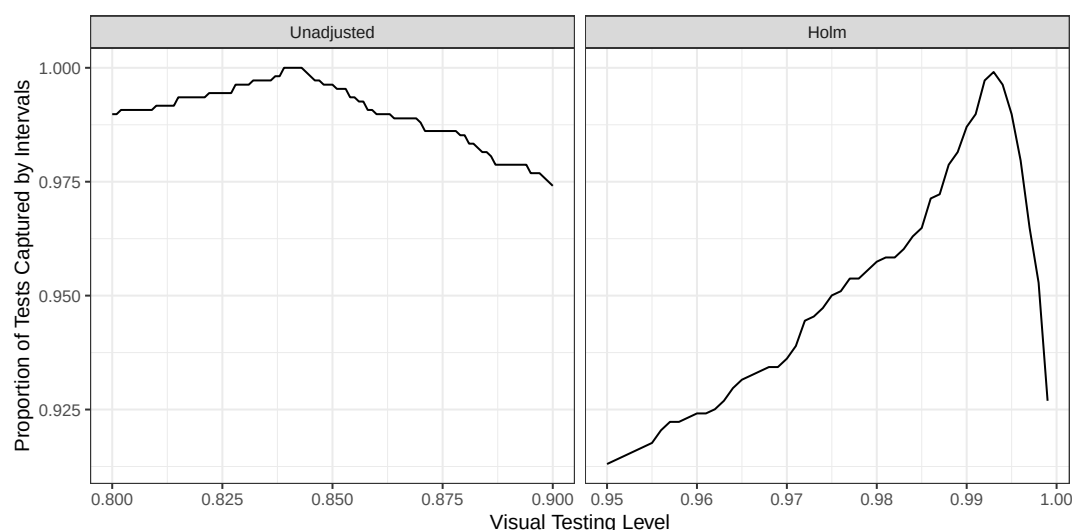


Figure 11: Proportion of Pairwise Tests Captured by Visual Testing Intervals Using a High Resolution Grid. The unadjusted grid uses 101 points between .8 and .9 clearly identifying the optimal level. The Holm adjustment uses 50 levels from .95 to .999 also identifying the optimal value.

6 Discussion

The examples above provide a lot of detail on how the package works and how users might discuss the findings in their own work. However, there are a few topics that may be of interest that were not covered above. We turn to some brief discussions of those below.

6.1 What if it doesn't work

Perhaps one of the most important issues is what happens when the function doesn't work. The `viztest()` function always produces an answer. However, as we showed, sometimes no level corresponds perfectly with the pairwise tests. How the user proceeds likely depends on their goals and how many (or maybe what proportion) of the tests are missed. If only a few tests are missed, users may decide to just discuss those discrepancies in the text or the note of the corresponding figure. If many tests are missed, perhaps another method would be preferred, like one of the heat maps. We do not think it makes sense to propose a strict cutoff above which users should use a different method. It really comes down to the ease of discussing the missed cases and the importance of presenting the intervals. If there are some estimates that are less important, users may consider dropping the less important estimates to see if that helps.

When there is no single level that works, breaking estimates into groups, if possible, could work. Having more than one group of estimates and allowing each group to have its own inferential confidence level could generate perfect correspondence within groups, but will necessarily leave across-group comparisons indeterminate. If there are natural groupings of estimates, this could be a useful solution. One example of this would be like our Iyengar and Westwood replication. In their article, there was a three-category treatment variable – Democratic candidate most qualified, Republican candidate most qualified and both candidates equally qualified. Had the algorithm not produced a single perfect interval, we could have broken the estimates into treatment groups and then focused on within-treatment tests.

Another thing to consider is whether tests relative to zero are included. Including tests against zero (or any point null) imposes an extra set of constraints that may make finding an appropriate level more difficult. If the user also includes standard confidence intervals (e.g., 95%), then those tests against a point null (including zero) can be accomplished by considering the overlap of the standard confidence intervals with the null hypothesis value. The inferential confidence intervals would then only have to capture the pairwise tests of estimates relative to each other.

Finally, one edge case where confidence intervals get strange is with data in categorical choice models that have perfect separation – no variance on the dependent variable for some category of an independent variable (Firth, 1993; Zorn, 2005). In these cases, the coefficients tend toward the boundaries of the parameter space (e.g., very large positive or negative values). They generally have very large standard errors and confidence intervals that are extremely wide. Below is a simple example of what these models can look like:

```
#>
#> Call:
#> glm(formula = y ~ grp, family = binomial, data = sep_data)
#>
#> Coefficients:
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)  -18.57    1304.53  -0.014    0.989
#> grpB           17.81    1304.53   0.014    0.989
#> grpC           18.81    1304.53   0.014    0.988
#> grpD           18.97    1304.53   0.015    0.988
#> grpE           19.32    1304.53   0.015    0.988
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#> Null deviance: 170.97  on 124  degrees of freedom
#> Residual deviance: 130.63  on 120  degrees of freedom
#> AIC: 140.63
#>
#> Number of Fisher Scoring iterations: 17
```

On the parameters themselves, `viztest()` does not work especially well. It tries to make the intervals really small and misses one test.

```
#>
#> Correspondents of PW Tests with CI Tests
#>   level   psame   pdiff   easy method
#> 1  0.25 0.9333333 0.0666667 -795.3725 Lowest
#> 2  0.62 0.9333333 0.0666667 -2261.0741 Middle
#> 3  0.99 0.9333333 0.0666667 -6791.1112 Highest
#> 4  0.25 0.9333333 0.0666667 -795.3725 Easiest
#>
#> Missed Tests for Lowest Level (n=1 of 15)
#>   bigger smaller pw_test ci_olap
#> 3  grpE    grpB    Sig    Yes
#>
#> Missed Tests for Middle Level (n=1 of 15)
#>   bigger smaller pw_test ci_olap
#> 3  grpE    grpB    Sig    Yes
#>
#> Missed Tests for Highest Level (n=1 of 15)
#>   bigger smaller pw_test ci_olap
#> 3  grpE    grpB    Sig    Yes
#>
#> Missed Tests for Easiest Level (n=1 of 15)
#>   bigger smaller pw_test ci_olap
#> 3  grpE    grpB    Sig    Yes
```

However, on the level of predictions, it seems to work better. We make predictions on the link scale to produce a situation where normal theory tests make most sense. In this case, despite the fact that one prediction has a very large variance, the algorithm accounts for all tests. Even in these stranger cases, the algorithm performs as expected on the predictions even if it struggles somewhat on the model parameters themselves. One of the big differences is that the parameters have very strong inter-correlations, but the predictions do not.

```
#>
#>   grp Estimate Std. Error      z Pr(>|z|)  S    2.5 %  97.5 %
#>   A  -18.566   1304.528 -0.0142  0.9886 0.0 -2.58e+03 2.54e+03
#>   B   -0.754     0.429 -1.7581  0.0787 3.7 -1.59e+00 8.66e-02
#>   C    0.241     0.403  0.5985  0.5495 0.9 -5.49e-01 1.03e+00
#>   D    0.405     0.408  0.9932  0.3206 1.6 -3.95e-01 1.21e+00
#>   E    0.754     0.429  1.7581  0.0787 3.7 -8.66e-02 1.59e+00
#>
#> Type: link
```

```
#>
#> Correspondents of PW Tests with CI Tests
#>   level psame pdiff      easy method
#> 1  0.84     1   0.1 -0.28590407 Lowest
#> 2  0.88     1   0.1 -0.03223582 Middle
#> 3  0.92     1   0.1 -0.29973546 Highest
#> 4  0.88     1   0.1 -0.03223582 Easiest
#>
#> All 10 tests properly represented for by CI overlaps.
```

6.2 Stability of the results – does sample size or variance matter?

One reviewer asked about the stability of these results (i.e., do sample size or variances matter)? The answer, we suppose, is “probably.” Since the input to the function is generally the output from a model (or other quantities derived from a model), the sample size and sampling variability will at least matter insofar as they matter for those estimates. We do not see a strong systematic relationship between sample size and the ability of the function to find an inferential confidence interval that corresponds with the pairwise tests. This correspondence will depend a lot more on the pattern of the significant differences, which does not depend on sample size directly. It could be the case that the estimate for a single small group with high variance may be difficult to balance with other estimates of smaller sampling variability, but this will depend on the values of the estimates, too.

6.3 Tests and miscellaneous issues

The function has several checks to ensure inputs are appropriate. The number of estimates has to be the same as the number of rows in the variance-covariance matrix. These checks will prevent the user from submitting incompatible estimates and variance-covariance matrices. Further, there is a check for symmetry and positive-definiteness of the variance-covariance matrix. If these checks fail, the function will return an error. If you suspect the error is because of floating-point issues, there is a tolerance that can be tweaked via the `tol` argument. We have also implemented a series of unit tests that ensure the function works as expected under a variety of scenarios.

There are two edge cases that may result in unexpected results. The first is when all estimates are statistically different from each other. In this case, the lower bound of acceptable intervals will be whatever the lower bound of `range_levels` is. Making the confidence bounds increasingly smaller will increase the contrast of the non-overlapping nature of the intervals. Picking a level that makes the differences clear, but without making the intervals unnecessarily small would be a good fix. Likewise, if no estimates are statistically different from any others, the upper bound of the acceptable range will be the upper bound of `range_levels`. Making the intervals increasingly larger will increase the contrast on the overlapping nature of all intervals. Again, using a value that identifies the overlap, but does not make the intervals unnecessarily large would be a good fix.

There is currently no mechanism to identify a priority in estimates whose differences are captured by the algorithm. One could imagine a situation where estimates of theoretical importance are given priority and the inferential confidence intervals are found such that those estimates are captured for certain and others are accommodated as is possible. One way of doing this would be to call `viztest()` on the important estimates first to find the range of acceptable levels. Then run `viztest()` on the full set of estimates and print the results using the levels identified in the initial run. This will identify the tests that are not appropriately captured by the inferential confidence level that works for the important estimates.

7 Conclusion

As [Armstrong II and Poirier \(2025\)](#) show, inferential confidence intervals can help solve the visual testing problem. By choosing the appropriate inferential confidence level, users can perform pairwise statistical tests accurately and reliably in most cases. The software described in this article allows users to calculate these inferential confidence levels in R for a wide class of objects using both normal-theory tests as well as tests on Bayesian posterior simulations. Users can interrogate these results with the package’s `print()` method and plot the results either with the package’s `plot()` method or via their graphing tool of choice.

References

- V. Arel-Bundock. *Models to Meaning: How to Interpret Statistical Models with R and Python*. Chapman & Hall, Boca Raton, FL, 2026. [p13]
- V. Arel-Bundock, N. Greifer, and A. Heiss. How to interpret statistical models using `marginaleffects` for R and Python. *Journal of Statistical Software*, 111(9):1–32, 2024. URL <https://doi.org/10.18637/jss.v111.i09>. [p5]
- D. A. Armstrong II and W. Poirier. Decoupling visualization and testing when presenting confidence intervals. *Political Analysis*, 33:252–257, 2025. URL <https://doi.org/10.1017/pan.2024.24>. [p1, 2, 10, 24]
- D. Bates, M. Mächler, B. Bolker, and S. Walker. Fitting linear mixed-effects models using `lme4`. *Journal of Statistical Software*, 67(1):1–48, 2015. URL <https://doi.org/10.18637/jss.v067.i01>. [p6]
- R. H. Browne. On visual assessment of the significance of a mean difference. *Biometrics*, 35(3):657–665, 1979. URL <https://doi.org/10.2307/2530259>. [p1]
- W. S. Cleveland and R. McGill. Graphical perception: Theory, experimentation and application to the development of graphical methods. *Journal of the American Statistical Association*, 79(387):531–554, 1984. URL <https://doi.org/10.1080/01621459.1984.10478080>. [p5]
- L. Dice and H. Laraas. A graphic method for comparing several sets of measurements. *Contributions from the Lab of Vertebrate Genetics*, (3):1–3, 1936. [p1]
- S. Few. *Show Me the Numbers: Designing Tables and Graphs to Enlighten*, 2nd ed. Analytics Press, Burlingame, CA, 2012. [p1]
- D. Firth. Bias reduction of maximum likelihood estimates. *Biometrika*, 80(1):27–38, 1993. URL <https://doi.org/10.1093/biomet/80.1.27>. [p22]
- J. Fox, S. Weisberg, and B. Price. *carData: Companion to Applied Regression Data Sets*, 2022. URL <https://CRAN.R-project.org/package=carData>. R package version 3.0-5. [p6]
- H. Goldstein and M. J. R. Healey. The graphical presentation of a collection of means. *Journal of the Royal Statistical Society, Series A*, 158(1):175–177, 1995. URL <https://doi.org/10.2307/2983411>. [p1]
- M. Gordon and T. Lumley. *forestplot: Advanced Forest Plot Using ‘grid’ Graphics*, 2025. URL <https://CRAN.R-project.org/package=forestplot>. R package version 3.1.7. [p6, 16]
- T. Hothorn, F. Bretz, and P. Westfall. Simultaneous inference in general parametric models. *Biometrical Journal*, 50(3):346–363, 2008. URL <https://doi.org/10.1002/bimj.200810425>. [p3]
- R. Iannone, J. Cheng, B. Schloerke, E. Hughes, A. Lauer, J. Seo, K. Brevoort, and O. Roy. *gt: Easily Create Presentation-Ready Display Tables*, 2025. URL <https://CRAN.R-project.org/package=gt>. R package version 1.0.0. [p3]
- J. P. Kastellec and E. L. Leoni. Using graphs instead of tables in political science. *Perspectives on Politics*, 5(4):755–771, 2007. URL <https://doi.org/10.1017/S1537592707072209>. [p1]
- R. V. Lenth. *emmeans: Estimated Marginal Means, aka Least-Squares Means*, 2025. URL <https://CRAN.R-project.org/package=emmeans>. R package version 1.11.1. [p3]
- M. Lu, J. Lanir, C. Wang, Y. Yao, W. Zhang, O. Deussen, and H. Huang. Modeling just noticeable differences in charts. *IEEE Transactions on Visualization and Computer Graphics*, 28(1):718–726, 2022. URL <https://doi.org/10.1109/TVCG.2021.3114874>. [p5]
- T. Lumley. *rmeta: Meta-Analysis*, 2018. URL <https://CRAN.R-project.org/package=rmeta>. R package version 3.0. [p6, 17]
- M. Meredith and J. Kruschke. *HDInterval: Highest (Posterior) Density Intervals*, 2022. URL <https://CRAN.R-project.org/package=HDInterval>. R package version 0.2.4. [p3]
- M. E. Payton, M. H. Greenstone, and N. Schenker. Overlapping confidence intervals or standard error intervals: What do they mean in terms of statistical significance? *Journal of Insect Science*, 3(1):34, 2003. URL <https://doi.org/10.1093/jis/3.1.34>. [p1]

- T. L. Pedersen. *patchwork: The Composer of Plots*, 2024. URL <https://CRAN.R-project.org/package=patchwork>. R package version 1.3.0. [p6]
- M. Radean. The significance of differences interval: Assessing the statistical and substantive difference between two quantities of interest. *Journal of Politics*, 85(3):969–983, 2023. URL <https://doi.org/10.1086/723999>. [p2]
- D. Robinson, A. Hayes, and S. Couch. *broom: Convert Statistical Objects into Tidy Tibbles*, 2025. URL <https://CRAN.R-project.org/package=broom>. R package version 1.0.10. [p3]
- J. M. Shea. *wooldridge: 115 Data Sets from "Introductory Econometrics: A Modern Approach, 7e" by Jeffrey M. Wooldridge*, 2024. URL <https://CRAN.R-project.org/package=wooldridge>. R package version 1.4-4. [p6]
- G. G. Simpson and A. Roe. *Quantitative Zoology, revised edition*. McGraw-Hill, New York, NY, 1939. [p1]
- J. Tukey. The philosophy of multiple comparisons. *Statistical Science*, 6(1):100–116, 1991. URL <https://doi.org/10.1214/ss/1177011945>. [p1]
- D. L. Veenstra, S. Saint, S. Saha, T. Lumley, and S. D. Sulliva. Efficacy of antiseptic impregnated central venous catheters in preventing nosocomial infections: A meta-analysis. *Journal of the American Medical Association*, 281(3):261–267, 1999. URL <https://doi.org/10.1001/jama.281.3.261>. [p17]
- H. Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2016. ISBN 978-3-319-24277-4. URL <https://ggplot2.tidyverse.org>. [p3]
- H. Wickham, R. François, L. Henry, K. Müller, and D. Vaughan. *dplyr: A Grammar of Data Manipulation*, 2023. URL <https://CRAN.R-project.org/package=dplyr>. R package version 1.1.4. [p3]
- H. Wickham, D. Vaughan, and M. Girlich. *tidyr: Tidy Messy Data*, 2024. URL <https://CRAN.R-project.org/package=tidyr>. R package version 1.3.1. [p3]
- C. Zorn. A solution to separation in binary response models. *Political Analysis*, 13(2):157–170, 2005. URL <https://doi.org/10.1093/pan/mpi009>. [p22]

William Poirier
Western University
Department of Political Science
London, Canada
<https://williampo1.github.io/>
ORCID: 0000-0002-3274-1351
wpoirier@uwo.ca

David A. Armstrong
Western University
Department of Political Science
London, Canada
<https://www.quantoid.ca/>
ORCID: 0000-0002-9358-2489
dave.armstrong@uwo.ca